PROGRAMMING FOR PROBLEM SOLVING (RCS101)

UNIT# 2

Module 2 (Arithmetic Expression & Conditional Branching)

Lecture 12:- Operators and expression using numeric and relational operators, logical Operators.

Lecture 13:- Relational operators Unary operator and assignment operators.

Lecture 14:- Ternary Operator, comma operator, increment/decrement operator

Lecture 15: Bitwise Operator

Lecture 16:- Type conversion, mixed Operands, operator precedence and associativity

Lecture 17: Conditional Branching: Applying if, nesting if & else

Lecture 18: Use of break and default with switch.

Lecture 19: Use of continue with example.

Lecture 20: Program for nested if

Lecture 21: Program for switch case.

Lecture 22:- Program for Operator uses.

Scanned with CamScanner

PROGRAMMING FOR PROBLEM SOLVING (KCS 101T)

Operators and Expression using numeric and relational operators, Logical Operators.

Lecture # 12 UNIT # 2

Operator and expression :

Operators and Operands create expression. These are predefined symbols. Each operator has special and unique meaning.

For example:- $\boxed{C = a + b}$

- C → Storage Operand
- a → first Operand
- b → Second Operand
- $+$ → Binary Plus operator.

Hence, operator is a symbols that tells the compiler to perform specific mathematical or logical functions.

Types of Operator :-

C language provides the following types of operator :-

Scanned with CamScanner

1) Unary Operator

★ (which applied on single Operand only)

★ Are of following type

1. Increment ++
2. Decrement --
3. Unary minus -
4. Unary plus +

3) Logical Operator

Logical AND &&

Logical OR ||

Logical Not !

2) Binary Operator

★ (which applied on two Operands)

★ Are of following type

1) Arithmetic Operator

- Addition +
- Subtraction -
- Multiplication *
- Division /
- Modulus %

2) Relational Operator

less than <

Less than equal to <=

Greater than >

Greater than equal to >=

Double equal to ==

Not equal to !=

3) Ternary Operator

★ (which applied on three Operands)

Only one operator in this :-

?:

↑

Conditional Operator also.

4) Bitwise Operator

1. Bitwise AND &
2. Bitwise OR |
3. Bitwise X-OR ^
4. Bitwise left shift <<
5. Bitwise Right shift >>
6. Bitwise Complementation ~

5) Assignment Operator

=

+=

-=

*=

/=

%=

Scanned with CamScanner

UNARY Operator

Unary operator requires only one operand.

for example:- $+x, -y$ etc.

(ii) Increment Operator (++)

Increment Operator is used to increase the current value by 1.

Syntax

$++x$

Increment — variable name

(Pre increment)

First increase by one and then print

Example

if $i=100$

print $++i$

print $= 101$

$x++$

Variable Name — Increment

(Post Increment)

Firstly print and then increase by one.

if $i = 100$

print $i++$

print 100

then value of $i = 101$

(iii) Decrement Operator (--)

Decrement Operator is used to decrease the current value by 1.

Scanned with CamScanner

Syntax

$\underbrace{--}_{\text{Decrement Operator}}\underbrace{x}_{\text{VariableName}}$

(Pre Decrement)

first decrease by 1 and then use/print.

$x--$

(Post Decrement)

first use/print the decrease by 1.

(iii) **Unary Minus and plus**

Operator 1	Operator 2	result
+	+	+
−	−	+
+	−	−
−	+	−

for example

```
#include <stdio.h>
void main()
{
int a=-5, b;
b= -(a)
printf("%d", b);
}
```

Execution process

a = -5, b

b = -(a)

= -(-5)

$\boxed{b = +5}$

Scanned with CamScanner

BINARY Operator

which applied on two Operands.

(i) Arithmetic Operator

S.No.	Arithmetic Operator	Operation Name	Example A B.	Representation
1	+	Addition	3 + 2 = 5	A+B
2	−	Subtraction	4 − 1 = 3	A−B
3	*	Multiplication	4 * 3 = 12	A*B
4	/	Division	5/4 = 1.25	A/B
5	%	Modulus	5%4 = 1	A%B
			−4%5 = −4	
			4%−5 = 4	

Note :- In modulus if numerator is less than denominator so numerator becomes answer (No need to find remainder) only we check the sign of Numerator answer.

(ii) Relational Operator :(Comparison)

It is the operator which is used to compare two operands and shows the relation between two operands. It will always produce true (1) and false (0).

Scanned with CamScanner

S.No.	Operators	Example	Description
1	< less than	$x<y$	3<4, condition true, it will return 1
2	<= less than equal to	$x<=y$	3<=4, condition true, return 1.
3	> Greater than	$x>y$	3>4, condition false, return 0.
4	>= Greater than equal to	$x>=y$	3>=4, condition false, return 0
5	== equal to	$x==y$	3==4, return 0.
6	!= Not equal to	$x!=y$	3!=4, condition true, return 1.

value of x = 3 and value of y = 4

(3) LOGICAL OPERATOR

This class of C Operators is used to take decisions. There are 3 such operator. They are AND, OR and NOT. The first two are binary operator and the last one is an unary operator. The result of these operator is true or false.

assume x = 5
y = 7

S.No.	Operators	Example	Description	Meaning
1	&&	(x>6) && (y>4)	(5>6) && (7>4) F T F = 0	LOGICAL AND
2	||	(x>=9) || (y>=10)	(5>=9) || (7>=10) F || F F = 0	LOGICAL OR
3	!	!(x>6)	!(5>6) = !F !(0) = 1	LOGICAL NOT

Scanned with CamScanner

illustrate the increment & decrement operator

```
# include <stdio.h>
# include <conio.h>
void main( )
{
 int a,b;
 printf ("enter the value of a,b");
 Scanf ("%d %d", &a, &b);
 printf ("%d ", a++); || (first print then increment)
 printf ("%d", ++a); || first increment then print
 printf ("%d", b++); first print then increment
 printf ("%d", b--); first print then decrement
 printf ("%d", --b); first decrement then print.
 getch();
}
```

O/P

```
a=4, b=5
4
6
5
6
4
```

Scanned with CamScanner

Illustrates the basic arithmetic operations :-

```
#include <stdio.h>
main()
{
 int n1, n2;
 int sum, diff, prod, quotient, remainder;
 printf (" Enter the n1, n2");
 scanf ("%d %d", &n1, &n2);
 sum = n1 + n2;
 diff = n1 - n2;
 prod = n1 * n2;
 quotient = n1 / n2;
 remainder = n1 % n2;
 printf ("sum = %d", sum);
 printf ("diff = %d", diff);
 printf ("prod = %d", prod);
 printf ("quotient = %d", quotient);
 printf ("remainder = %d", remainder);
 getch();
}
```

Scanned with CamScanner

PROGRAMMING FOR PROBLEM SOLVING (KCS-101T)

Relational Operator, Unary Operator and assignment Operator.

Lecture #13 UNIT #2

Relational Operator and Unary Operator already discussed in previous lecture.

ASSIGNMENT OPERATOR

Assignment operator have two values :- L-value and R-value Operator copies R-value into L-value. It is denoted by equal to and add another symbols such as

	Operators		Example	Explanation
1)	Simple Assignment Operator	=	a = 10	10 is assigned to variable a
2)	Compound	+=	C += 10	C = C+10
		-=	C -= 10	C = C-10
3)	Assignment	*=	CS *= 10	CS = CS * 10
		/=	CS /= 10	CS = CS/10
4)	Operators	%=	CS % = 10	CS = CS % 10
		&=	CS & = 10	CS = CS & 10
		^=	CS ^ = 10	CS = CS ^ 10

Scanned with CamScanner

Illustrates the Assignment Operator

```
# include <stdio.h>
# include <conio.h>
void main( )
{
 clrscr();
 int a, b, c;
 printf (" enter the value of a,b,c");
 Scanf ("%d %d %d", &a, &b, &c);
 a+=b ;
 b-=a ;
 c*=a ;
 printf ("a=%d", a);
 printf (" b=%d", b);
 printf ("c=%d", c);
 getch();
}
```

enter the value of a, b, c 4 5 6

Output :-

a=9

b = 5-9 = -4

C= 6*9

=54

Scanned with CamScanner

PROGRAMMING FOR PROBLEM SOLVING (KCS-101T)

Ternary Operator, comma operator, increment/decrement operator.

Lecture #14 UNIT-2

Ternary (Conditional) Operator :-

It is the Operator that uses three arguments such as

$$\boxed{\text{Expression 1 ? expression 2 : expression 3}}$$

True

false.

where expression 1 is the condition. If the condition is true then expression 2 will be result otherwise operand 3 will be result.

for example:-

```
# include <stdio.h>
void main ()
{
int a=5, b=7, c;
c= a>b ? a:b;
printf ("%d", c);
}
```

Execution process

a=5, b=7, c

c = a>b ? a:b

= 5>7 ? 5 : 7

T

F

false

$\boxed{c = 7}$

d

d = x > c ? x : c

Special Operator :-

1. &
2. Comma Operator
3. sizeof

1. Address Operator (&)

This is used to get the address of variable (&a)

2. Comma Operator

The Comma Operator is basically associated with for statement. It is also used to link two or more related expressions together. In such a case, the expression are evaluated in a left to right fashion. And we can obtain the value of the combined expression from the value of the rightmost expression.

Example:-

```
# include <stdio.h>
# include <conio.h>
void main()
{
  int a, b, temp
  b = (a=20, a+10);
  printf ("a =%d and b = %d (before)\n", a, b);
  temp = a, a = b, b = temp;
  printf ("a =%d and b=%d (after)\n", a, b);
}
```

Output

a = 20 and b = 30 (before)
a = 30 and b = 20 (after

(5) **Size of( ) Operator**

Sizeof( ) Operator returns the size of the Operand. It should be written in lower case letters. It must precede its operand. The Operand may be constant, variable or datatype. Normally used to determine the size of arrays & structures.

Syntax Sizeof (Operand);

Example

```
x = sizeof (int);
y = sizeof (sum);
```

Note:- Increment and Decrement Operator already discussed in previous lecture.

Scanned with CamScanner

PROGRAMMING FOR PROBLEM SOLVING (KCS-101T)

Bitwise Operator

Lecture #15 UNIT #2

Bitwise Operator:- Bitwise Operator uses the binary digits or bits.

Types of Bitwise Operator:

1. Bitwise AND operator (&)
2. Bitwise OR operator (|)
3. Bitwise XOR operator (^)
4. Bitwise ones compliment Operator (~)
5. Bitwise left shift operator (<<)
6. Bitwise Right Shift Operator (>>)

1. Bitwise AND Operator:-

In this Operator if both operands are true then only it will true.

Truth Table:-

Operand 1	Operand 2	Result
0	0	0
0	1	0
1	0	0
1	1	1

Scanned with CamScanner

for Example

```
#include <stdio.h>
#include <conio.h>
void main()
{
  int x=7, y=2;
  z=(x&y);
  printf("%d",z);
}
```

Execution process :-

$$x = 111$$
$$y = 010$$
$$x \& y = 010 = 2$$

$\boxed{C=2}$ Ans

② Bitwise OR Operator

The bitwise OR operator is represented by a single vertical sign(|). Two integer operands are written on both side of the (|) symbol. If the bit value of any of the operand 1, then the output would be 1, otherwise 0.

for example:-

Operand 1	Operand 2	Result
0	0	0
0	1	1
1	0	1
1	1	1

Scanned with CamScanner

for example :-

```
#include <stdio.h>
#include <conio.h>
void main()
{
int d=6, c=7, b;
b=(d|c);
printf("%d", b);
}
```

Execution Process

d = 6, c = 7, b

b = (d | c);

b = ?

d = 6 = 110

c = 7 = 111

d | c = 111 = 7

b = (d | c) = 7

(3) Bitwise XOR Operator

Bitwise exclusive OR operator is denoted by (^) symbol. If the corresponding bit of any of the operands is 1 then the output would be 1, otherwise 0.

Truth Table

Operand 1	Operand 2	Result
0	0	0
0	1	1
1	0	1
1	1	0

Scanned with CamScanner

For example

```
# include <stdio.h>
# include <conio.h>
void main()
{
  int x = 5 , y = 7 , z;
  z = (x ^ y);
  printf ("%d", z);
}
```

Execution process

$x = 5 \quad y = 7 \quad z = ?$

$z = (x \wedge y)$

$z = ?$

$x = 5 = 101$

$y = 7 = 111$

$x \wedge y = 010 = 2$

$\boxed{z = (x \wedge y) = 2}$

(4) Bitwise Complement Operator

The bitwise complement Operator is also known as one's complement Operator. It is represented by (~). It takes only one operand or variable and performs complement operation on an operand. When we apply complement operation on any bits, then 0 becomes 1 and 1 becomes 0.

Scanned with CamScanner

for example:-

```
#include <stdio.h>
#include <conio.h>
void main()
{
  int a=5;
  printf("%d",(~a));
}
```

Note:- Editor represent two's compliment.

a=5 = 101 = 010 + 1 = 011

Execution process

a = 5

~a = ~(5+1) = ~(6) = -6 Ans

0110

(5) Bitwise left shift Operator:-

Bit pattern of the data can be shifted by specified number of positions to left.

for example:-

```
#include <stdio.h>
#include <conio.h>
void main()
{
  int a=60, b;
  b = (a<<2)
  printf("%d", b);
}
```

Note:- Add two zero to the right

Execution process

a = 60 = 0000 0000 0011 1100

b = a<<2 ⇒ 0000 0000 1111 0000

b = 11110000

b = 240

Scanned with CamScanner

(6) Bitwise Right Shift Operator :-

Bitwise is a bit pattern of data, which can be shifted by specified number of positions to right.

For example :-

```
#include<stdio.h>
#include <conio.h>
void main()
{
  int a=60, b;
  b= (a>>2);
 printf("%d", b);
}
```

Execution process

a = 60 ⟹ 0000 0000 0011 1100

b = (a>>2) ⟹ 0000 0000 0000 1111

b = 1111

$\boxed{b = 15}$

Scanned with CamScanner